

The Shift to NetDevOps and the "As-Code" Philosophy

Executive Summary

For decades, network engineering has been defined by the Command Line Interface (CLI). It is fast, familiar, and highly effective for localized troubleshooting and configuration. However, as enterprise networks evolve into highly distributed, software-defined architectures (like Cisco SD-WAN), the traditional method of manually configuring devices—or even manually clicking through centralized GUIs—has become a bottleneck.

To scale securely, the networking industry is adopting the disciplines of software development. This shift, often called NetDevOps or Infrastructure-as-Code (IaC), replaces manual keystrokes with machine-readable definition files. This document outlines the history, mechanics, and operational benefits of the "As-Code" philosophy, serving as a foundational overview for SDWAN-as-Code methodologies.

The Origins: The DevOps Methodology

To understand Infrastructure-as-Code, it is necessary to look at the problem it was originally created to solve in the software world.

Historically, software companies suffered from a massive divide between two silos: Development and Operations. Developers were incentivized to push new features rapidly, while Operations teams were incentivized to keep servers stable, which often meant resisting change. When new code was handed over to operations, it frequently conflicted with existing environments, causing outages and operational friction.

To resolve this, the industry created DevOps—a methodology designed to merge these silos. Strict, automated disciplines were introduced so development could move quickly without compromising operational stability:

- **Version Control:** Every single change is tracked, timestamped, and attributed to a specific author.
- **Peer Review:** No code goes into production without being reviewed and approved by another engineer.
- **Automated Pipelines:** Code is programmatically tested and deployed, removing human error from the release process.

The "Wedge": Adapting Software Tools for Physical Networks

Infrastructure-as-Code (IaC) is the networking industry's attempt to adopt these exact DevOps disciplines. However, it is crucial to acknowledge that networking is not software, and these tools are not a perfect fit.

Software is ephemeral; if an application crashes, it can be terminated and a new instance spun up in seconds. Networks are physical and stateful; a core router cannot simply be rebooted, nor can a WAN link be dropped, without severe consequences.

Because of this, applying DevOps to networking requires adapting—or "wedging"—these methodologies into traditional workflows. It requires accepting certain gaps. For example, GitHub was designed to store application source code, not the routing state of a global enterprise network. Yet, it is adapted for this use because the benefits of having a version-controlled history of network configurations far outweigh the friction of using a tool slightly outside its original design.

The Technical Shift: Imperative vs. Declarative

At the core of the As-Code philosophy there is a fundamental shift in how administrators communicate with the network. It requires moving from an imperative mindset to a declarative mindset.

The Imperative Approach (Traditional CLI)

Traditional networking is imperative. The engineer dictates exactly how to achieve a goal, step-by-step. For example, to configure a BGP neighbor, a script is written:

```
router bgp 65000
  neighbor 10.1.1.2 remote-as 65001
  neighbor 10.1.1.2 description WAN-Uplink
```

The flaw in the imperative approach is that it is blind to the current state of the device. If this script is executed twice, it might throw an error, or it might blindly overwrite a change made by another administrator.

The Declarative Approach (As-Code)

Infrastructure-as-Code is declarative. The desired final state of the network is declared in a human-readable text file (typically YAML or JSON).

```
bgp_neighbors:
  - ip_address: "10.1.1.2"
    remote_as: 65001
    description: "WAN-Uplink"
    state: "present"
```

This file is handed to an automation engine (such as Terraform). The engine checks the current state of the network, compares it to the file, and calculates the difference. If the router is missing the description, the engine only adds the description. This is known as idempotency—the ability to run the same code repeatedly and achieve the exact same, safe result.

4. The Modern Workspace: The IDE

In the traditional CLI world, an engineer's primary tools are a terminal emulator (like SecureCRT or PuTTY) and a text editor (like Notepad++).

In the As-Code world, the primary workspace is an Integrated Development Environment (IDE). Currently, Visual Studio Code (VS Code) has emerged as the industry standard. It acts as a text editor, a terminal, and a connection to version control all in one window. It automatically color-codes YAML/JSON files, catches syntax errors before files are saved, and streamlines the process of writing and reviewing infrastructure code.

5. Operational Benefits: Why the Shift is Necessary

Despite the learning curve, the return on investment fundamentally improves network operations:

- **The Code is the Documentation:** In traditional networking, documenting why a change was made requires opening a separate ticketing system or updating a Wiki page. Because these are separate steps, documentation frequently falls out of date. In an As-Code environment, Version Control systems (like Git) enforce documentation. A change cannot be saved without a "Commit Message"—a plain-English note attached permanently to that specific change (e.g., "Added BGP neighbor for new ISP at Site 42, per Ticket INC12345"). Furthermore, before deployment, the change goes through a "Peer Review" (or Pull Request). This process ensures the history of the files automatically becomes a living, perfectly accurate log of who changed the network, when, and exactly why.

- **A True Single Source of Truth:** The network's configuration lives in a centralized repository. To know how a site is configured, an engineer simply reads the code repository.
- **Fearless Troubleshooting (Instant Diffs):** When a network outage occurs, engineers can instantly compare the current code repository with previous versions to see exactly what changed, highlighted line-by-line.
- **Instant, Mathematical Rollbacks:** If a deployment causes an issue, the change can be reverted in the version control system. The automation engine recalculates the state and safely undoes the configuration.

6. Application to Cisco SD-WAN

Cisco SD-WAN abstracts the traditional CLI through vManage. However, manually navigating through centralized GUI screens to deploy multiple sites remains prone to human error and is difficult to audit at scale.

By treating SD-WAN as Code, vManage templates, localized policies, and site-specific variables are defined in text files using an IDE. These files are stored in Git, reviewed by peers, and pushed to vManage via tools like Terraform and REST APIs. This ensures that deployments across all sites are executed with mathematical consistency, paving the way for a more resilient, scalable network.